So you want to learn AI

Why AI in JS

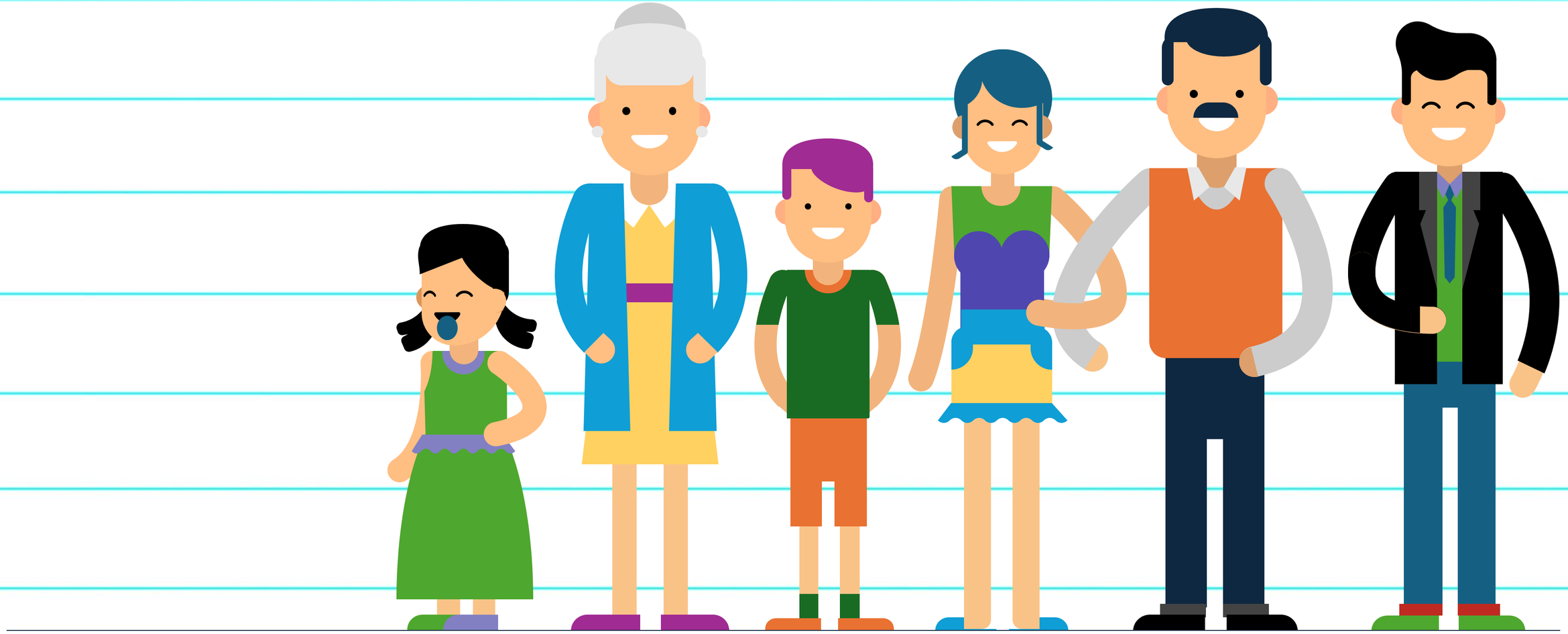
Ron Dagdag

Code > Create > Coach > Repeat.

R&D Engineering Manager at 7-ELEVEN

=

Generational Marketing



=

Generational Marketing

The Greatest Generation

Built resilience through the Great Depression and defined heroism in WWII.

1901–1924

The Silent Generation

Hardworking, disciplined, and shaped by post-war stability.

1925–1945

The Baby Boomer Generation

Grew up in prosperity, fueled cultural revolutions, and reshaped work and retirement.

1946–1964

Generation X

The independent, skeptical, latchkey kids who bridged analog and digital worlds.

1965–1979

Millennials

Tech-savvy, purpose-driven, and shaped by 9/11, social media, and the gig economy.

1980–1994

Generation Z

Digital natives, socially conscious, and redefining norms in work, activism, and identity.

1995–2012

Gen Alpha

touchscreen-first generation growing up in an always-connected world.

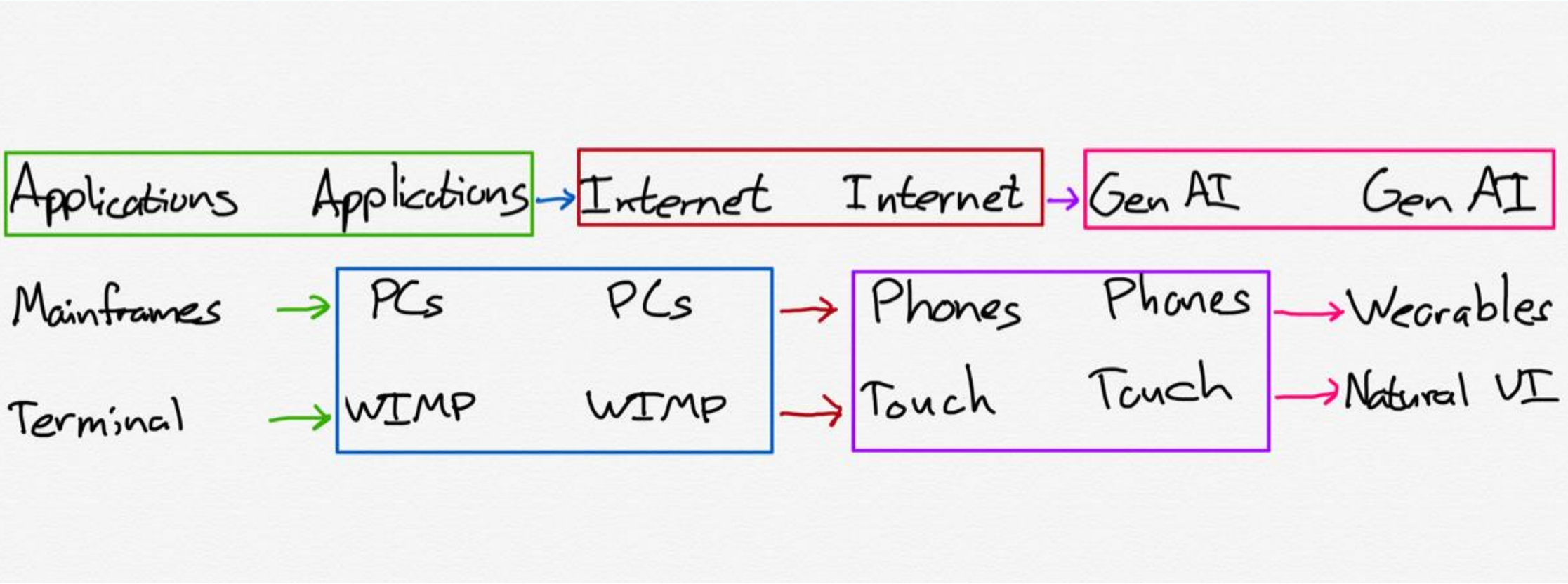
2013–2025

=

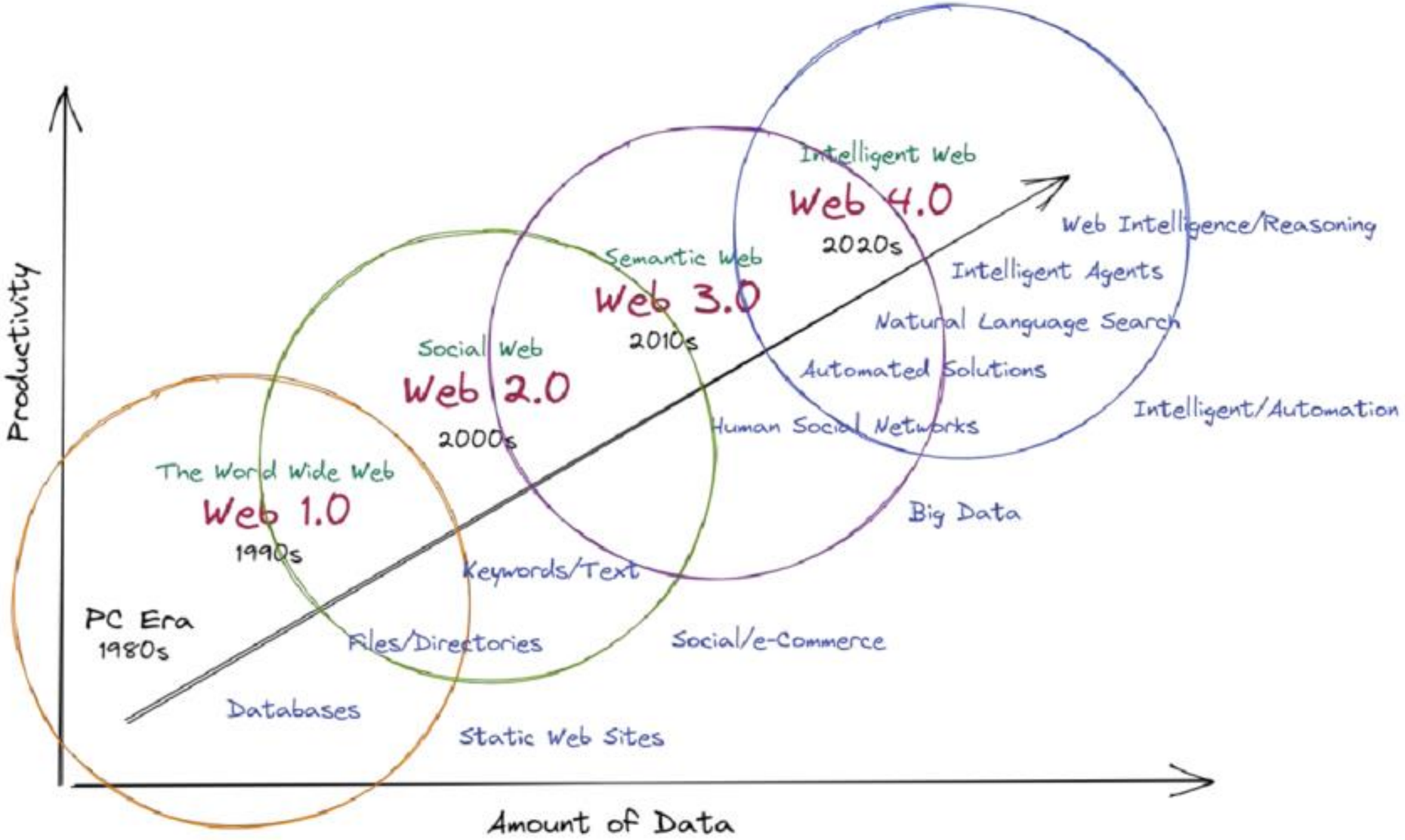
Developer Generations

Mainframe Generation (1950s-1970s)	Client-Server Generation (1980s-1990s)	Web Generation (1995-2005)	Mobile Generation (2006-2015)	Cloud Generation (2010-2020)	AI Generation (2020-Present)	Agentic Generation (2025-Future)
Developers who worked with **punch cards, COBOL, and Fortran** on massive centralized systems.	Developers building networked applications, databases, and enterprise systems using languages like **C, C++, and early Java.**	Developers who embraced **HTML, CSS, JavaScript, and PHP**, shaping the first wave of dynamic websites.	Developers who pioneered smartphone apps **with iOS, Android**, and cross-platform tools like **React Native.**	Developers focused on scalable, serverless, and distributed computing with **AWS, Azure, and DevOps** culture.	Developers working with **machine learning**, LLMs, and AI-powered applications, integrating intelligence into everyday software.	Developers focusing on **autonomous AI agents**, multi-agent systems, and self-improving software architectures .

Technology Bridges



Stratechery by Ben Thompson
https://stratechery.com/2024/the-gen-ai-bridge-to-the-future/

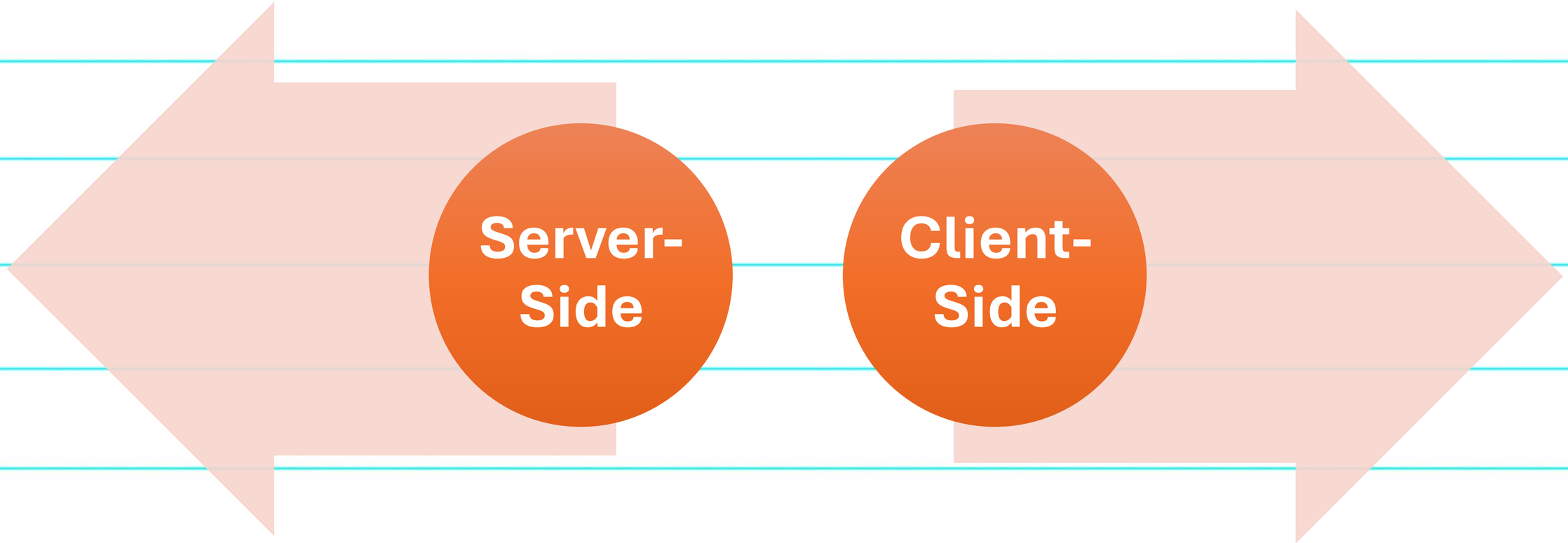


Evolution of the Web

Exploring the Power of Creative AI Tools and Game-Based Methodologies for Interactive Web-Based Programming - Benjamin Kenwright
https://arxiv.org/abs/2308.11649

=

Choose Sides



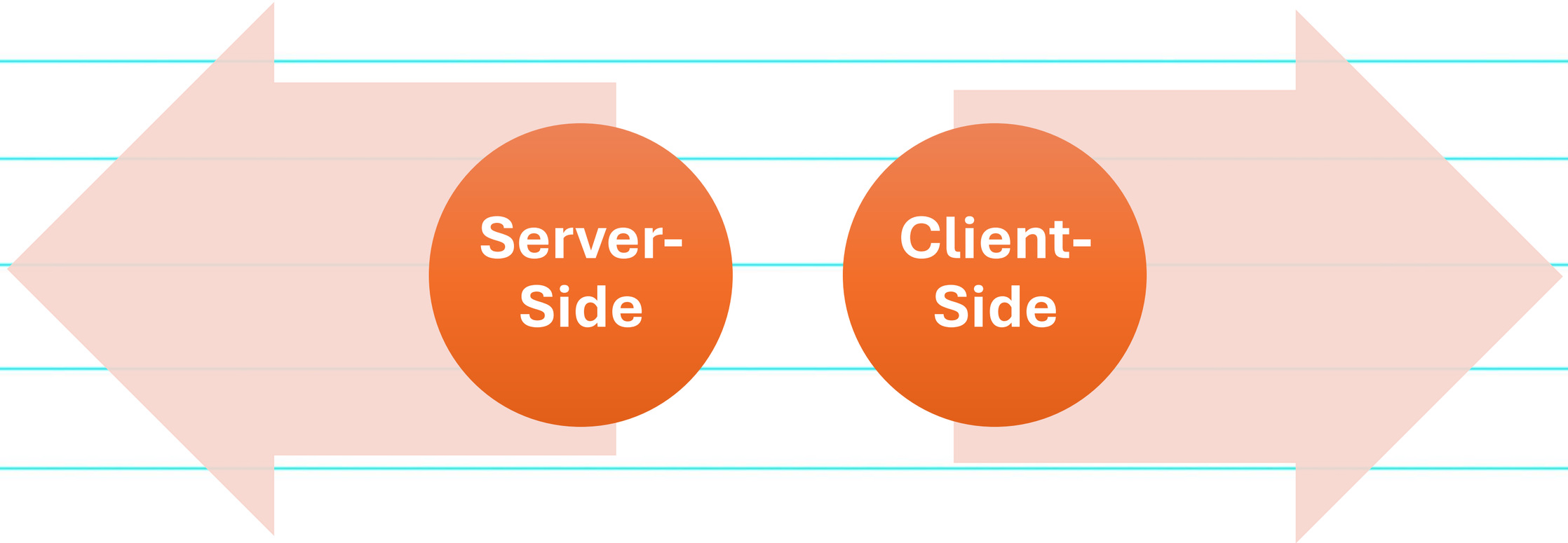
The Web Eats Everything in its Path

Graphics	Camera
Animation	Messaging
Location	Real-Time Messaging
Motion Input	IOT/Wearables
Real-Time 3D	Robotics

And now AI

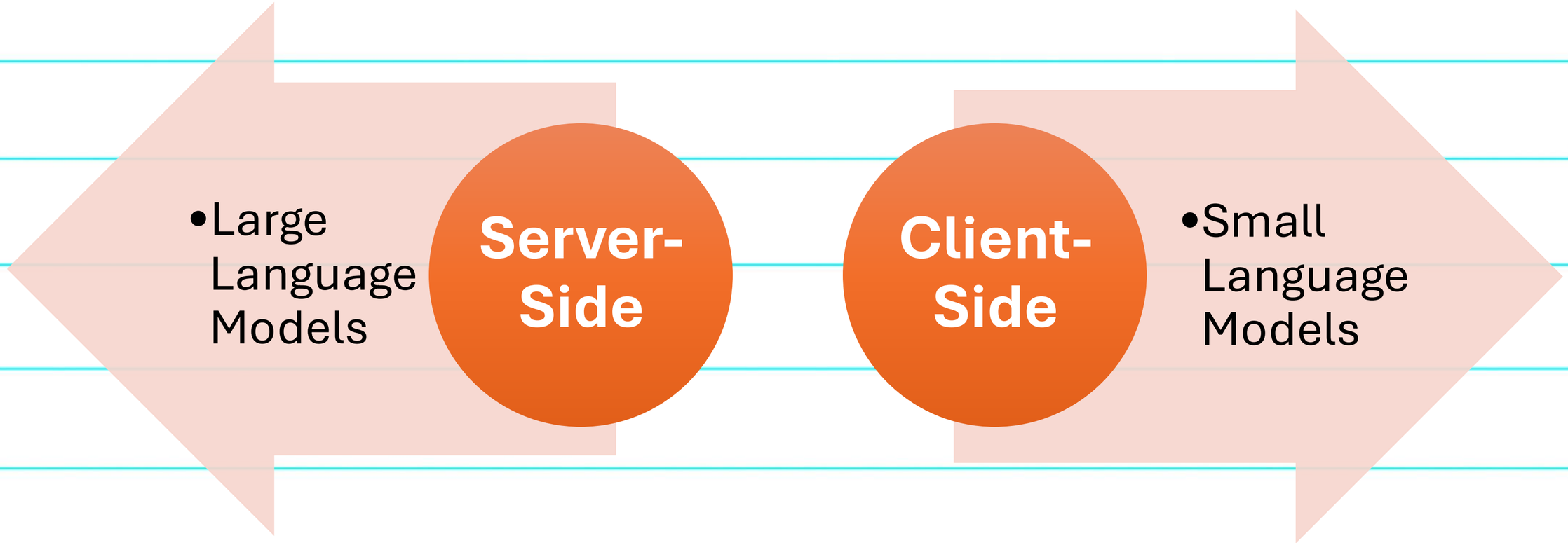
=

JavaScript



=

Foundation Models



=

Choose Sides

Server-Side

- Supports complex models
- Broader device compatibility
- High performance, scalable, continuous updates
- Leverages the Cloud AI ecosystem like Azure AI Foundry

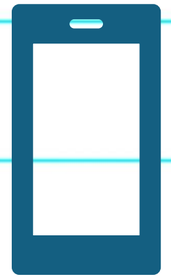
Client-Side

- Protects data, improves latency, offline support
- Purpose-built models for quick responses
- Model download or built-in browser AI

Hybrid AI Approach

Client-Side	• Handles specific, approachable tasks
Server-Side	• For complex models & broader device compatibility
Graceful Fallback	• Ensures functionality on older or less powerful devices

3 Approaches



On Browser / Device ML

Frontend AI via Cloud Models

Hybrid Implementation / Edge Enhanced

=

Server-Side JavaScript

NodeJS

- TensorFlow.js
- Transformers.js
- LangChain.js
- LlamaIndex.ts
- Brain.js
- ML.js
- ONNX.js

Server-Side

=

Why Server-Side AI?

- Supports complex models
- Broader device compatibility
- High performance, scalable, continuous updates
- Leverages the Cloud AI ecosystem like Azure AI Foundry

Benefits of Server-Side AI

Data Center Processing Power

Elastic scale

Centralized model updates

Access to shared, real-time data

Predictable experience across devices

=

LlamaIndex.TS

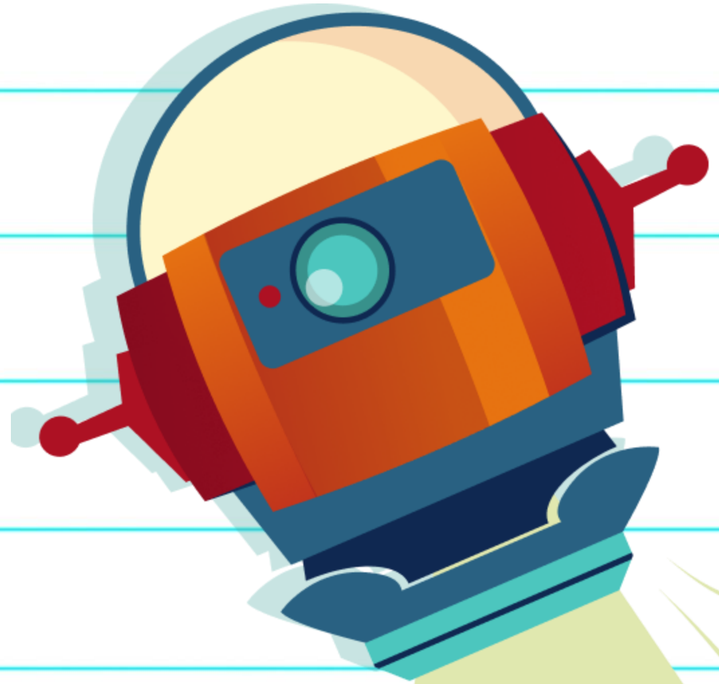
Agents

Build agentic RAG applications

Truly powerful retrieval-augmented generation applications use agentic techniques, and LlamaIndex.TS makes it easy to build them.

```
import { VectorStoreIndex } from "llamaindex";
import { SimpleDirectoryReader } from "@llamaindex/readers/directory";
import { openai } from "@llamaindex/openai";
import { agent } from "@llamaindex/workflow";

// load documents from current directoy into an index
const reader = new SimpleDirectoryReader();
const documents = await reader.loadData(currentDir);
const index = await VectorStoreIndex.fromDocuments(documents);

const myAgent = agent({
  llm: openai({ model: "gpt-4o" }),
  tools: [index.queryTool()],
});

await myAgent.run('...');
```

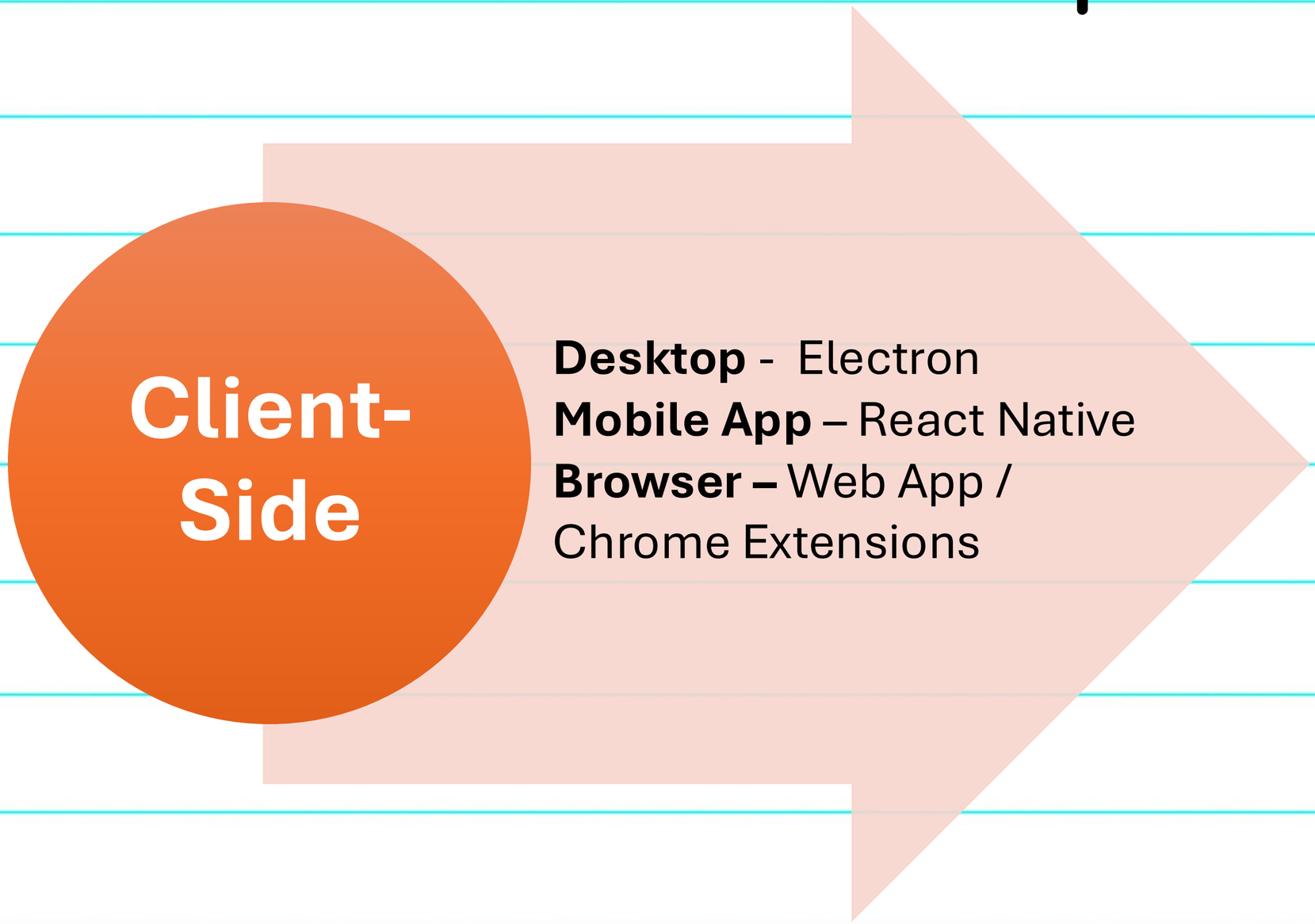
=



DEMO

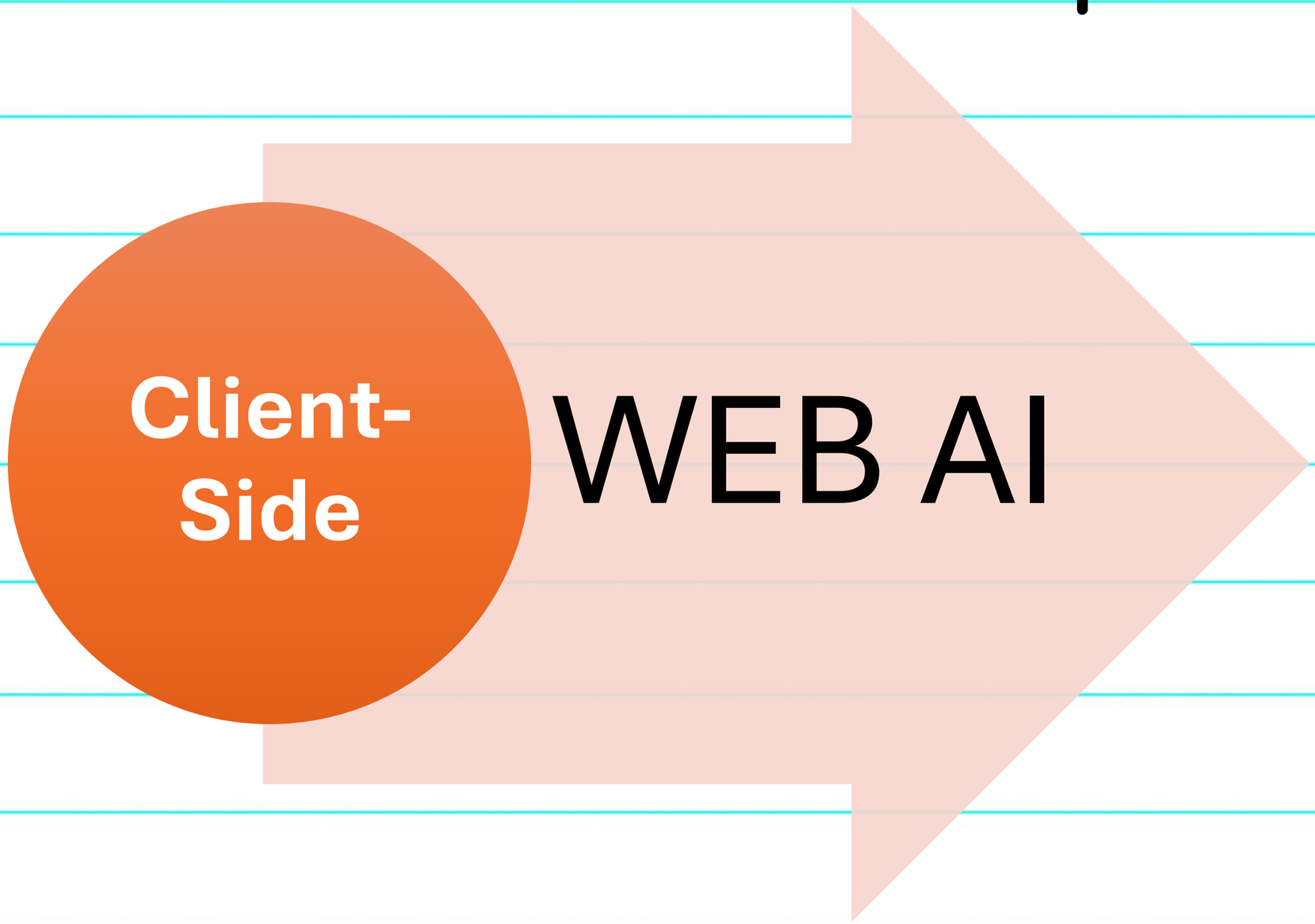
=

Client-Side Javascript



=

Client-Side Javascript



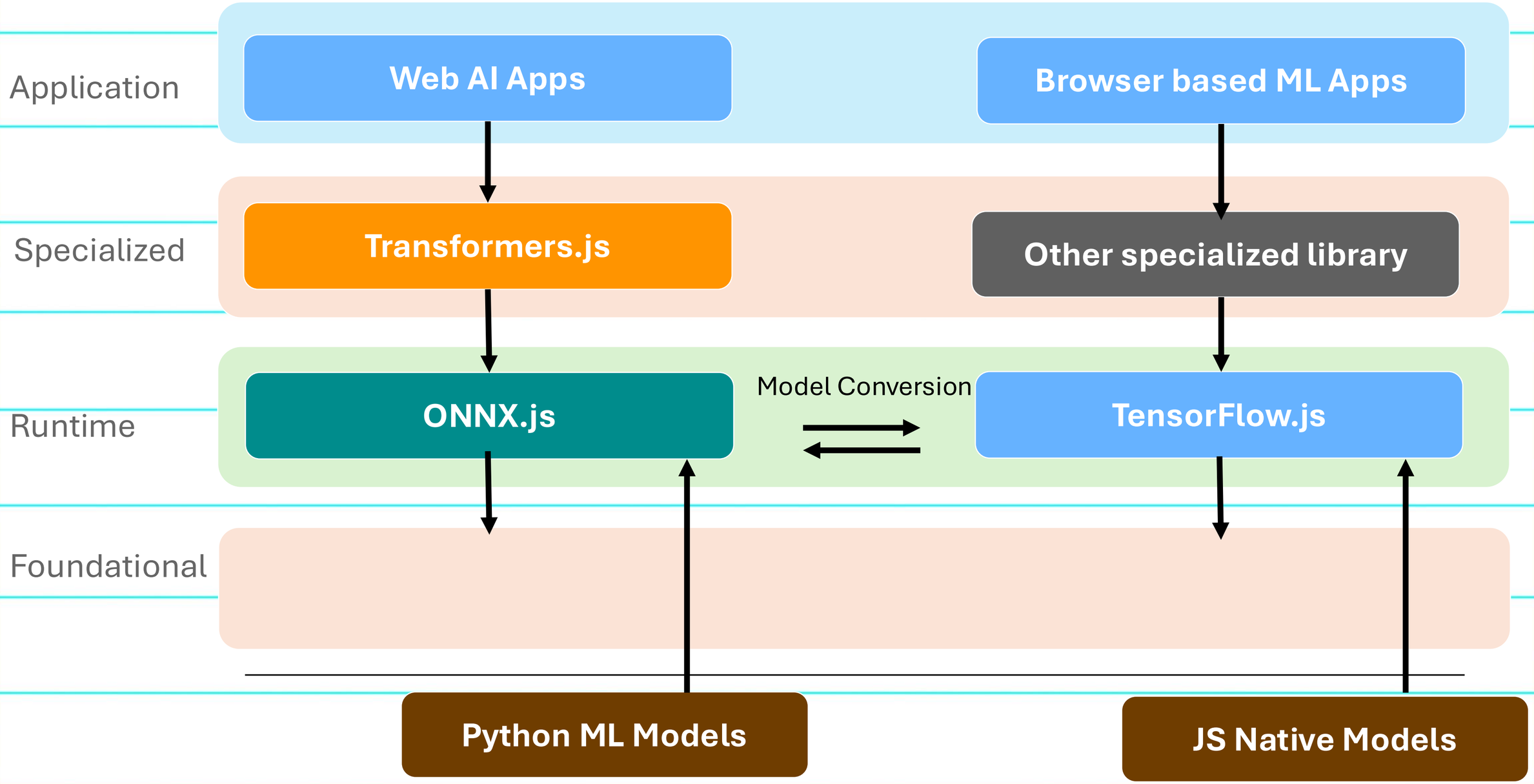
=

Why Client-Side AI?

- Integration with React & Angular
- Cross platform compatibility
- Offline capability
- Real-Time Processing
- Privacy and Security

=

Tech Stack


Application
Specialized
Runtime
Foundational
Web AI Apps
Browser based ML Apps
Transformers.js
Other specialized library
ONNX.js
Model Conversion
TensorFlow.js
Python ML Models
JS Native Models

=

Tensorflow.js

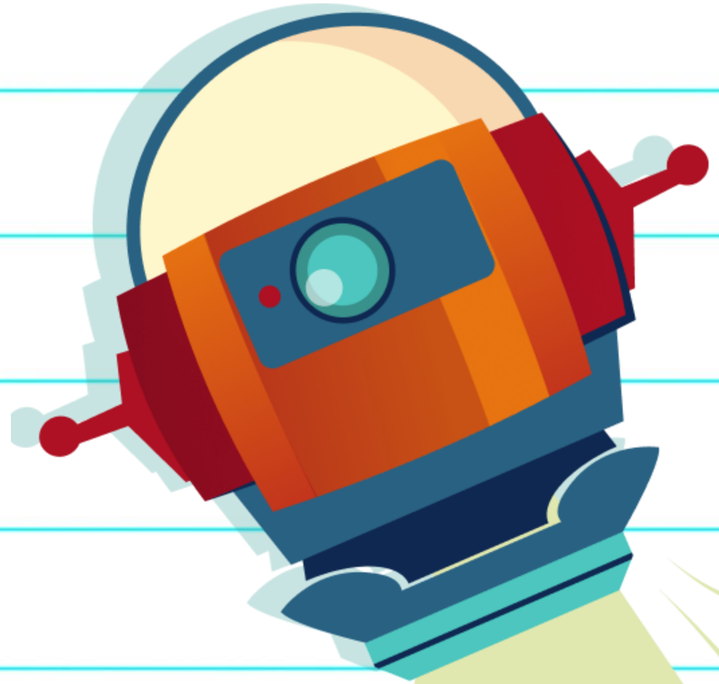
JavaScript library for training and deploying machine learning models

Runs directly in web browser and Node.js

No need for server-side execution or specialized hardware

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs"></script>
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/toxicity"></script>
    <script>
      const threshold = 0.5
      toxicity.load(threshold).then((model) => {
        const sentences = ["You are a tiny, little baby poop", "My favorite color is blue.", "Shut up!"];

        model.classify(sentences).then((predictions) => {
          console.log(JSON.stringify(predictions, null, 2));
        });
      });
    </script>
  </head>
  <body>
    <h1>Check the console log!</h1>
  </body>
</html>
```

=



DEMO

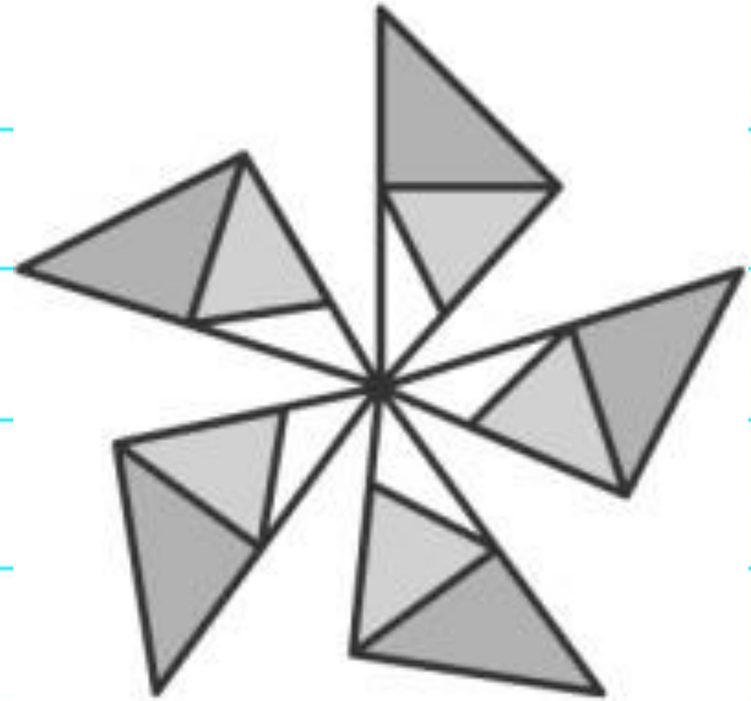
=

ONNX

Open Neural Network Exchange (ONNX) is an open standard for ML models

Enables interoperability across various open source AI frameworks

Offers flexibility in adopting different AI frameworks



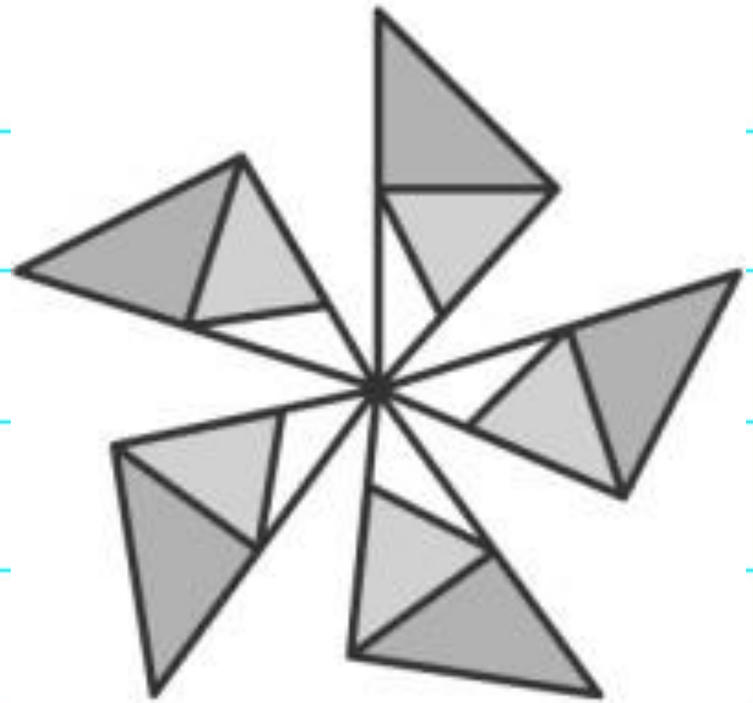
=

ONNX Runtime Web

A JavaScript library for running ONNX models

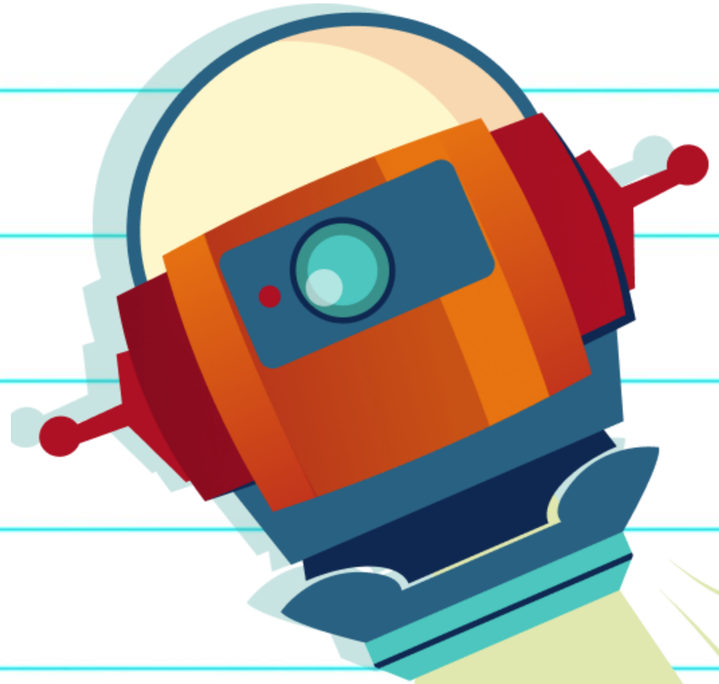
Supports both browser and Node.js environments

WebAssembly and WebGL



```
    <script src="https://cdn.jsdelivr.net/npm/onnxruntime-web/dist/ort.min.js"></script>
    <script>
        // use an async context to call onnxruntime functions.
        async function main() {
            try {
                // create a new session and load the specific model.
                //
                // the model in this example contains a single MatMul node
                // it has 2 inputs: 'a'(float32, 3x4) and 'b'(float32, 4x3)
                // it has 1 output: 'c'(float32, 3x3)
                const session = await ort.InferenceSession.create('./model.onnx');

                // prepare inputs. a tensor need its corresponding TypedArray as data
                const dataA = Float32Array.from([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]);
                const dataB = Float32Array.from([10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]);
                const tensorA = new ort.Tensor('float32', dataA, [3, 4]);
                const tensorB = new ort.Tensor('float32', dataB, [4, 3]);

                // prepare feeds. use model input names as keys.
                const feeds = { a: tensorA, b: tensorB };

                // feed inputs and run
                const results = await session.run(feeds);

                // read from results
                const dataC = results.c.data;
                document.write(`data of result tensor 'c': ${dataC}`);

            } catch (e) {
                document.write(`failed to inference ONNX model: ${e}.`);
            }
        }

        main();
    </script>
```

=



DEMO

Small Language Models

- Cost Effectiveness
- Deployment Flexibility
- Ultra-low Latency
- Easier to Customize

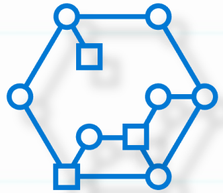
=

Phi Family Models

Small Language Models

Groundbreaking performance for size, with frictionless availability

Phi-3.5-mini
(3.8B)



Phi-3.5-vision
(4.2B)

Phi-3.5-MoE
(6.6B active)

Instruction Tuned

RAI Safety Aligned

Available on

Azure AI
Model Catalog

Hugging Face

ONNX Runtime

NVIDIA NIM

Ollama

Transformer.js

Run Transformers directly in your browser,
with no need for a server!

run pretrained models locally on your machine

https://huggingface.co/docs/transformers.js/index

https://huggingface.co/spaces/webml-community/phi-3.5-webgpu

https://huggingface.co/spaces/Xenova/doodle-dash

Transformer.js

- 📝 **Natural Language Processing**: text classification, named entity recognition, question answering, language modeling, summarization, translation, multiple choice, and text generation.

- 🖼️ **Computer Vision**: image classification, object detection, segmentation, and depth estimation.

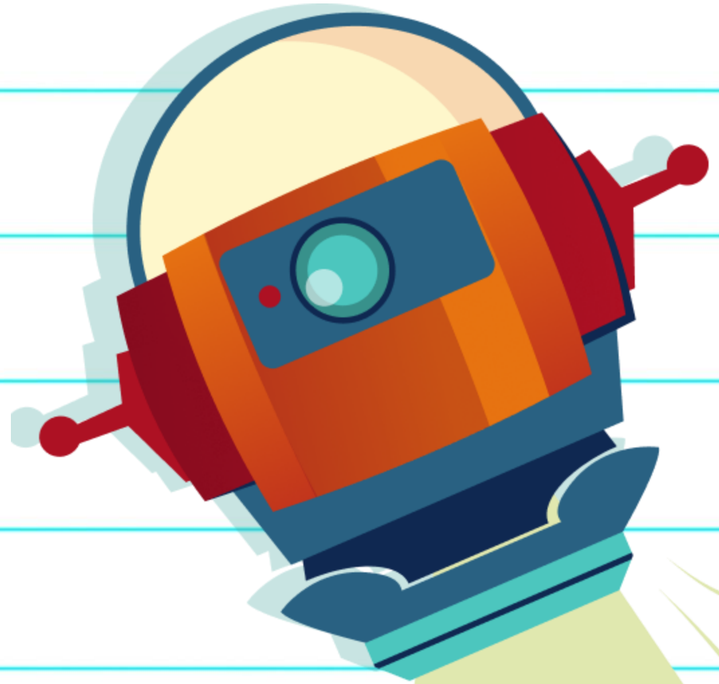
- 🗣️ **Audio**: automatic speech recognition, audio classification, and text-to-speech.

- 🐙 **Multimodal**: embeddings, zero-shot audio classification, zero-shot image classification, and zero-shot object detection.

Transformer.js

```
import { pipeline } from '@huggingface/transformers';

// Allocate a pipeline for sentiment-analysis
const pipe = await pipeline('sentiment-analysis');

const out = await pipe('I love transformers!');
// [{'label': 'POSITIVE', 'score': 0.999817686}]
```

=



DEMO

Gemini Nano

two small models:

Nano-1

and the slightly more capable

Nano-2

which is meant to run offline.

unlocks new possibilities for AI agents - intelligent systems that can use memory, reasoning, and planning to complete tasks for you.

=

Built-in AI

- AI models built directly into web browsers
- Runs locally on devices, reducing dependency on servers
- Enhances privacy and performance

Built-in AI

Ease of Deployment	• Browser distributes & updates models • No need to manage large downloads
Hardware Acceleration	• Optimized for GPUs, NPUs, CPUs
Local Processing	• Improves privacy & speeds up responses
Offline Usage	• AI features available without internet

=

Built-in AI

Privacy & Security	• Sensitive data stays on device
Snappy User Experience	• Near-instant results without server round trips
Cost & Scalability	• Offloads processing to users' devices
Enhanced Feature Access	• Preview premium features with minimal cost

Built-in AI

API	Explainer	Web	Extensions	Chrome Status	Intent
Translator API	MDN	Chrome 138	Chrome 138	View	Intent to Ship
Language Detector API	MDN	Chrome 138	Chrome 138	View	Intent to Ship
Summarizer API	MDN	Chrome 138	Chrome 138	View	Intent to Ship
Writer API	GitHub	Origin trial	Origin trial	View	Intent to Experiment
Rewriter API	GitHub	Origin trial	Origin trial	View	Intent to Experiment
Prompt API	GitHub	In Origin trial	Chrome 138	View	Intent to Experiment
Proofreader API	GitHub	In EPP	In EPP	View	Intent to Prototype

=

Built-in AI

Not yet supported:

- Chrome for Android
- Chrome for iOS
- Chrome for ChromeOS

AI Chrome Extensions

Enhance Browsing

- Control and customize web content

Elevate User Experience

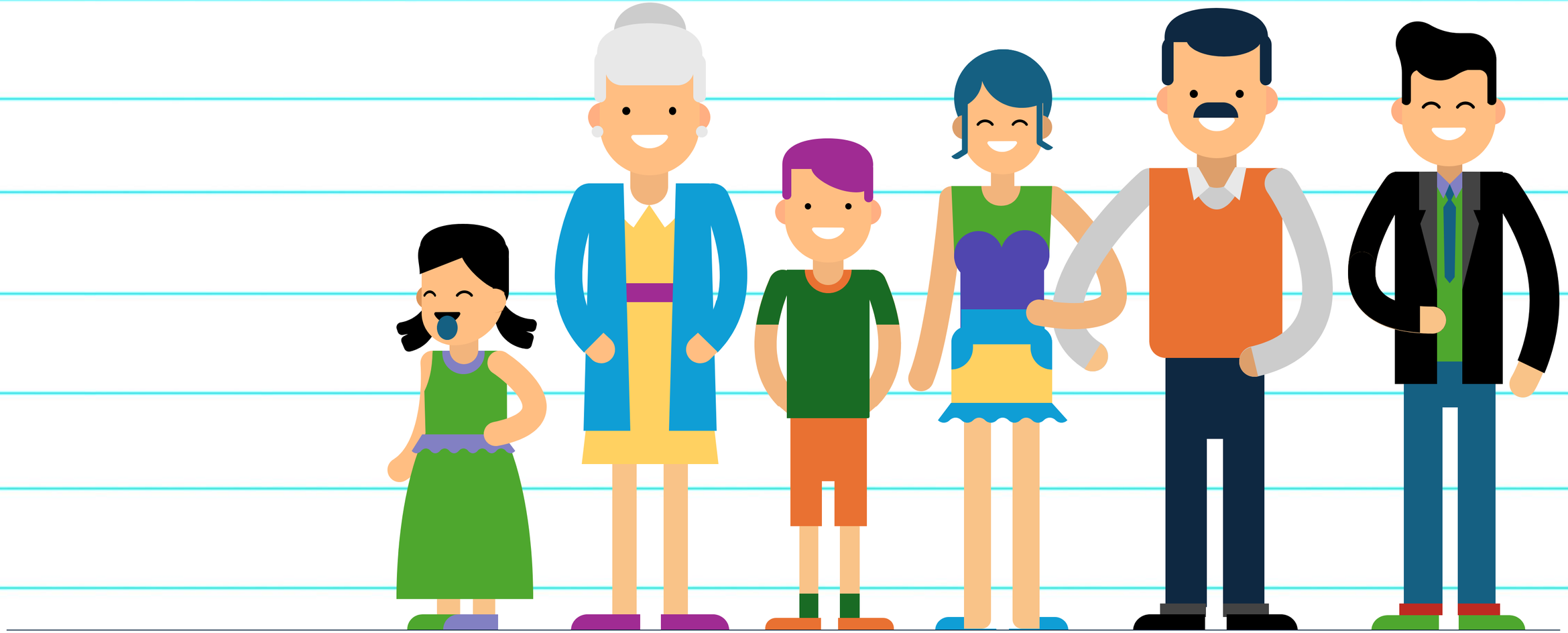
- Smarter bookmarks, new tab pages, history insights

Seamless Integration

- Side panels, action bars, context menus

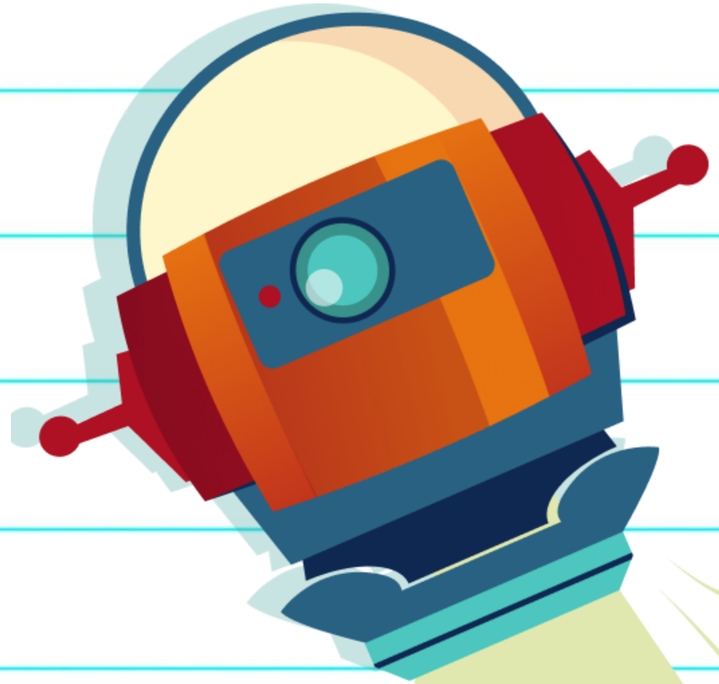
=

Explain in Generations



```
// Initialize the summarizer with current options
// @ts-expect-error new chrome feature
summarizer = await Summarizer.create(getOptions());
await summarizer.ready

// Process the selected text and stream the results
const stream = await summarizer.summarize(info.selectionText, {
  context: `article from ${new URL(tab.url!).origin}`
})
for await (const chunk of stream) {
  chrome.runtime.sendMessage({
    chunk,
    type: "STREAM_RESPONSE"
  })
}
```

=

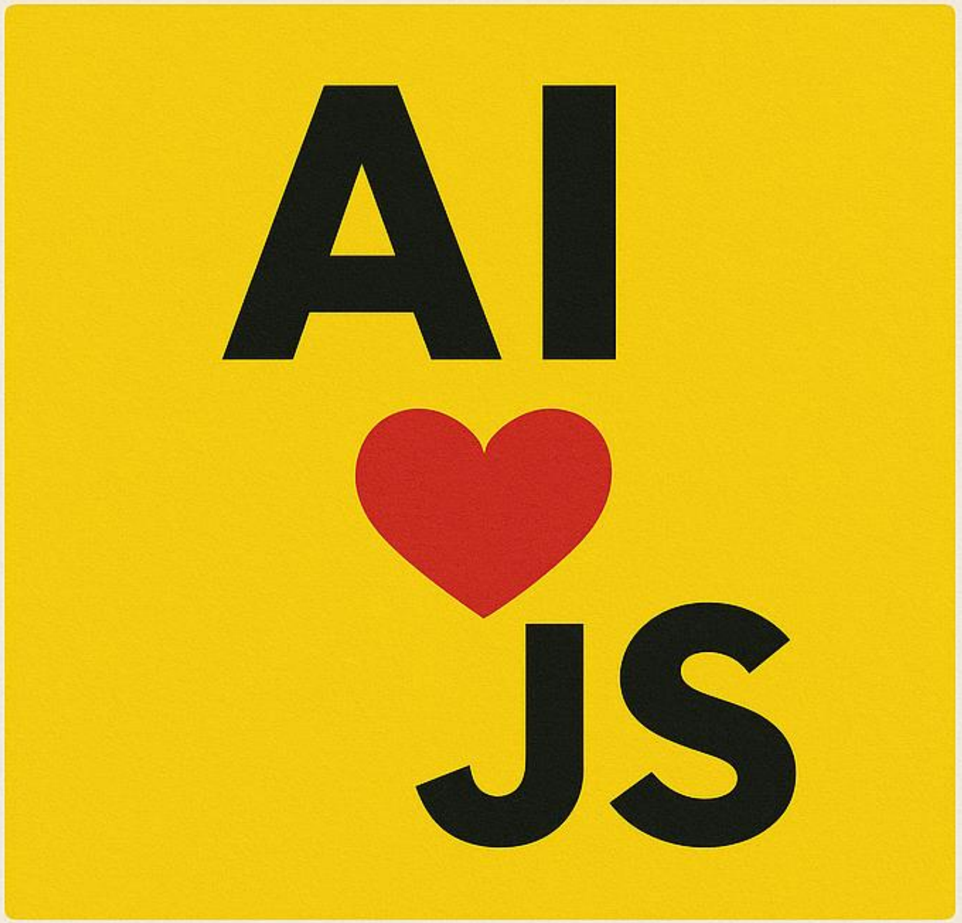


DEMO

=

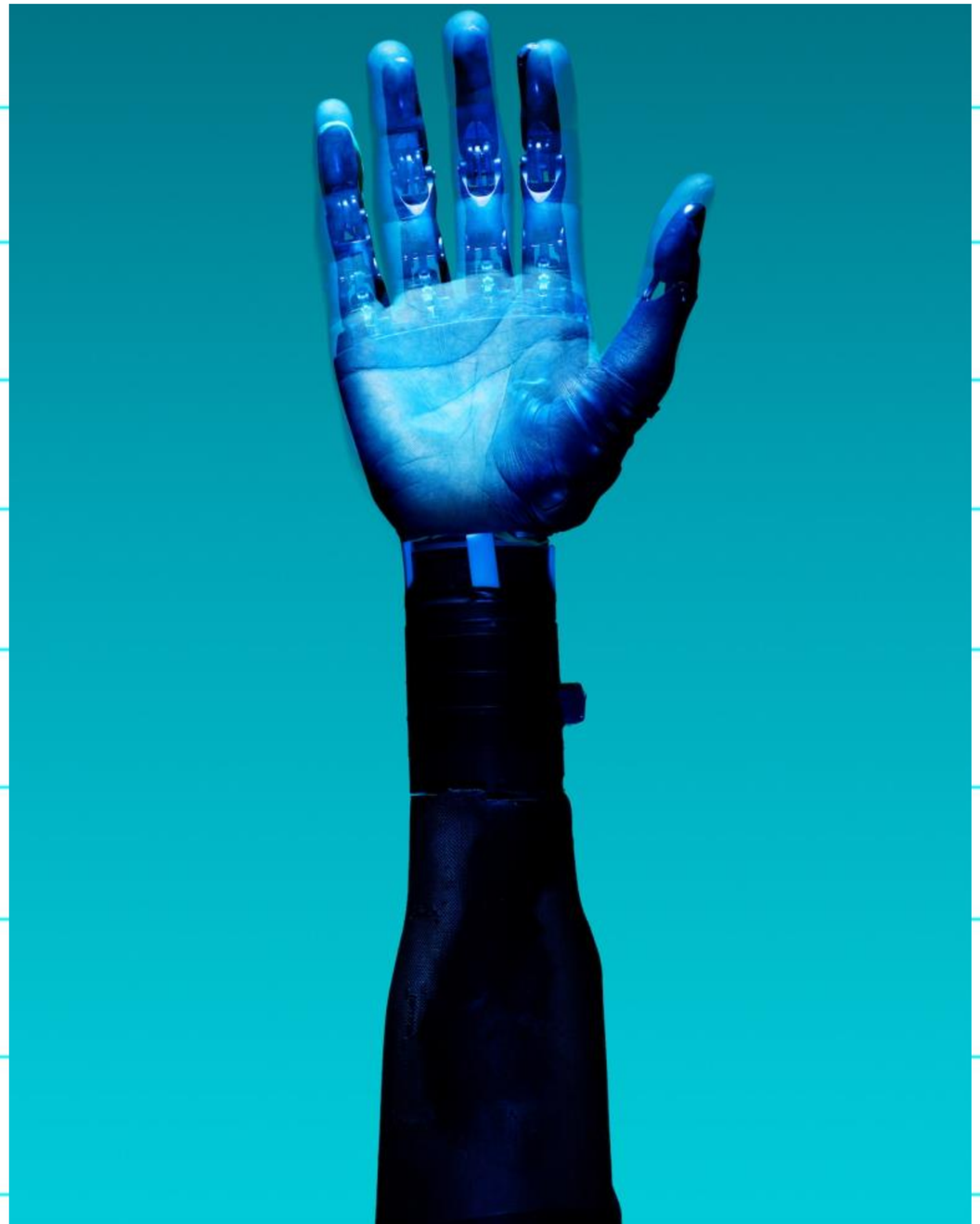
Why AI in JS

- Python remains the powerhouse for research & model training.
- Client-Side and Real-time, user-facing AI experiences run on JavaScript.
- JavaScript bridges complex models to seamless UX across web & mobile.
- Leverage existing JS skills to build AI-driven apps.
- Tools are ready.


AI
JS

=

Questions?



=

Ron Dagdag

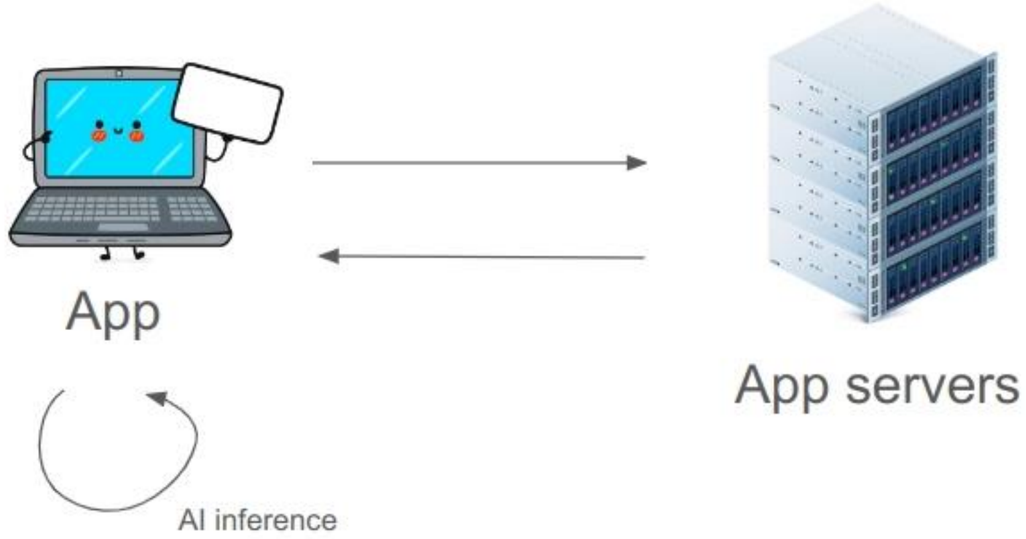
Microsoft® Most Valuable Professional

Award Categories
AI, Windows Development,
Internet of Things, Mixed Reality

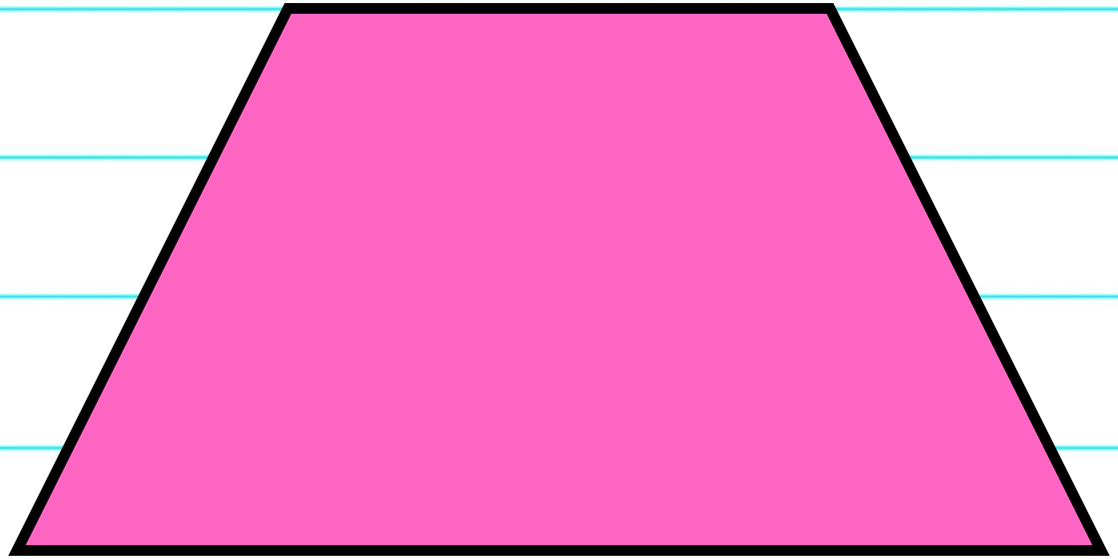
- R&D Engineering Manager at 7-Eleven
- 10th year Microsoft MVP awardee
- www.dagdag.net
- @rondagdag
- Linked In
  www.linkedin.com/in/rondagdag/
- Thanks for geeking out about AI in JavaScript

Then you look at getting started and see... Python. And math.

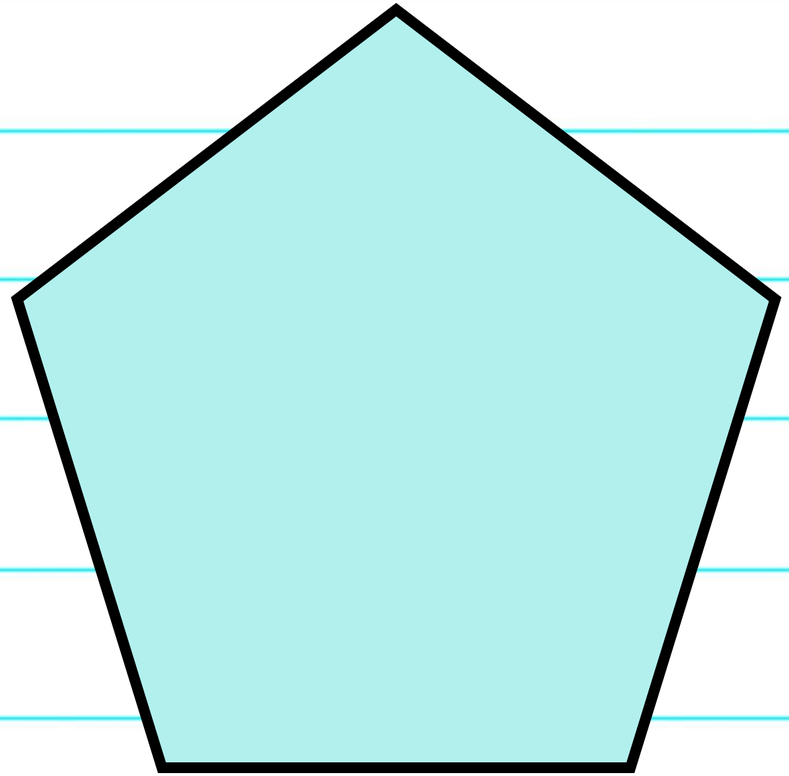
This is a heart.



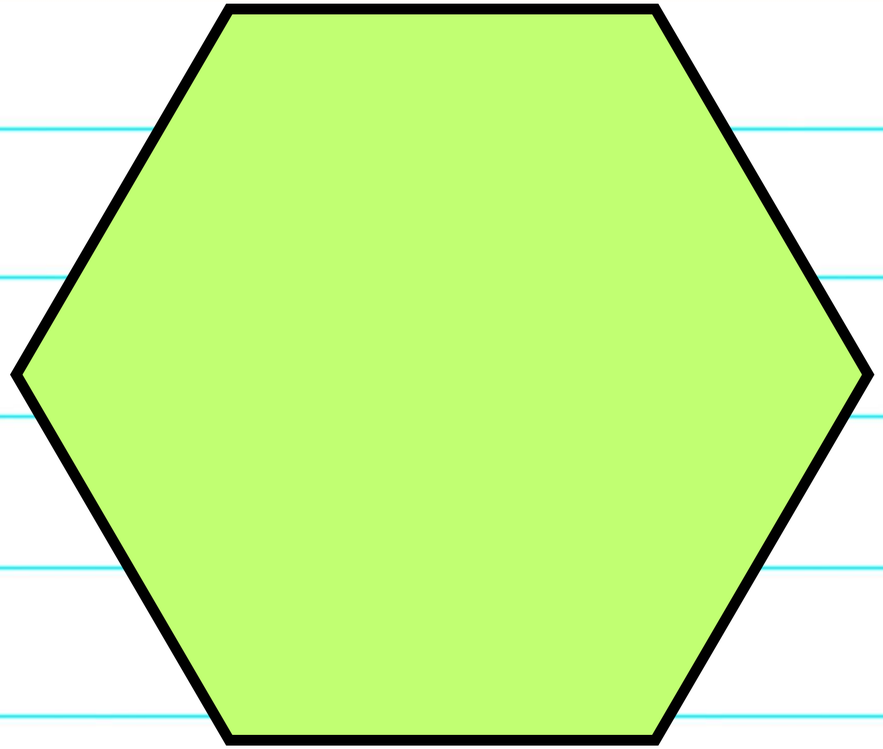
This is a heart.



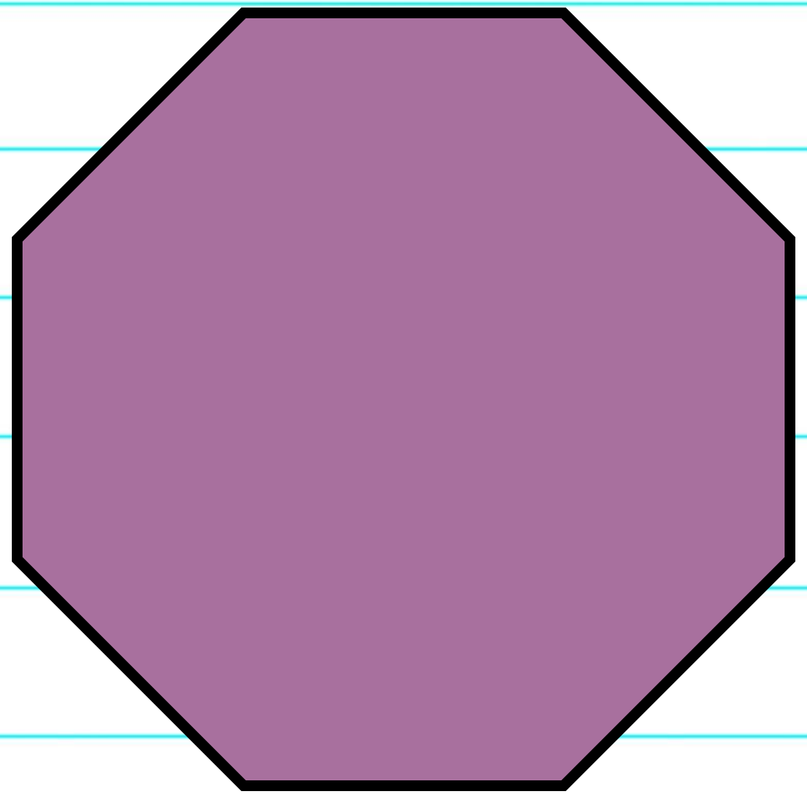
This is a trapezoid.



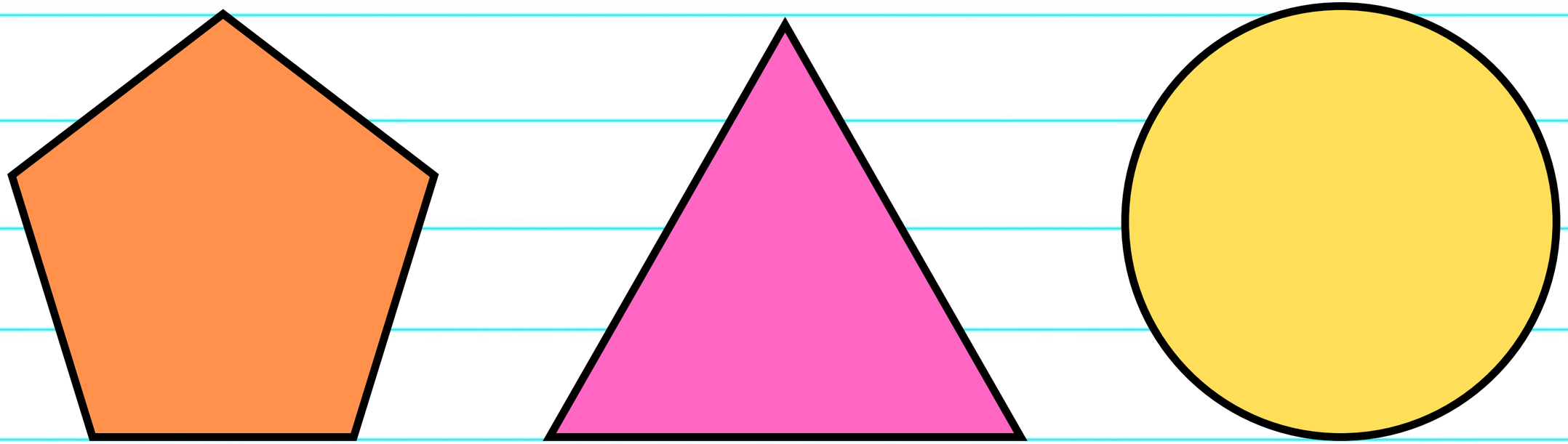
This is a pentagon.



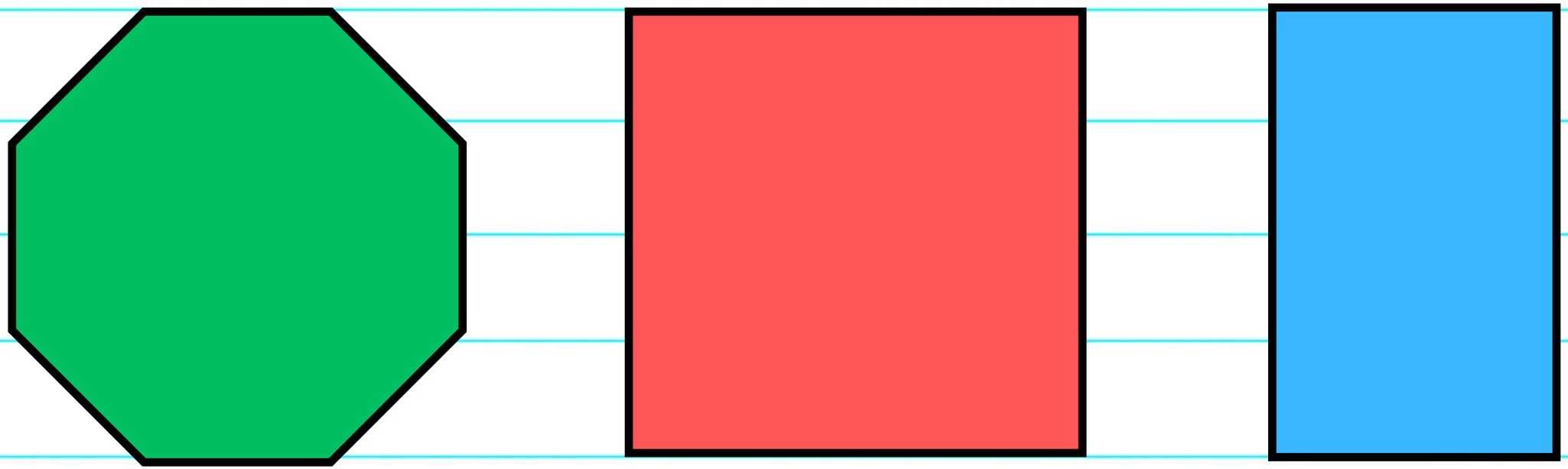
This is a hexagon.



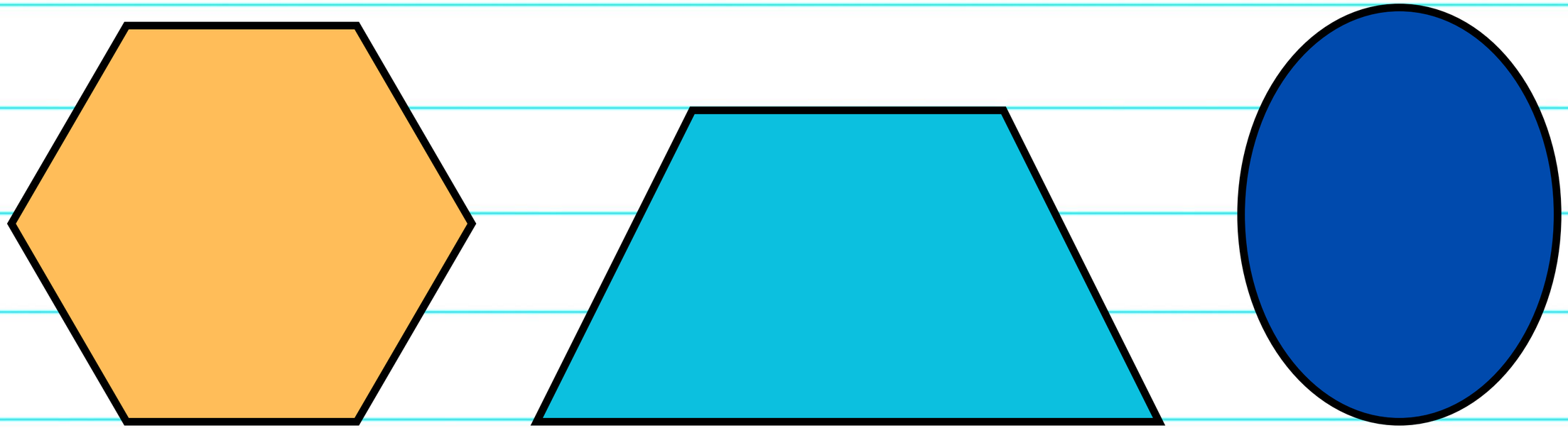
This is an octagon.



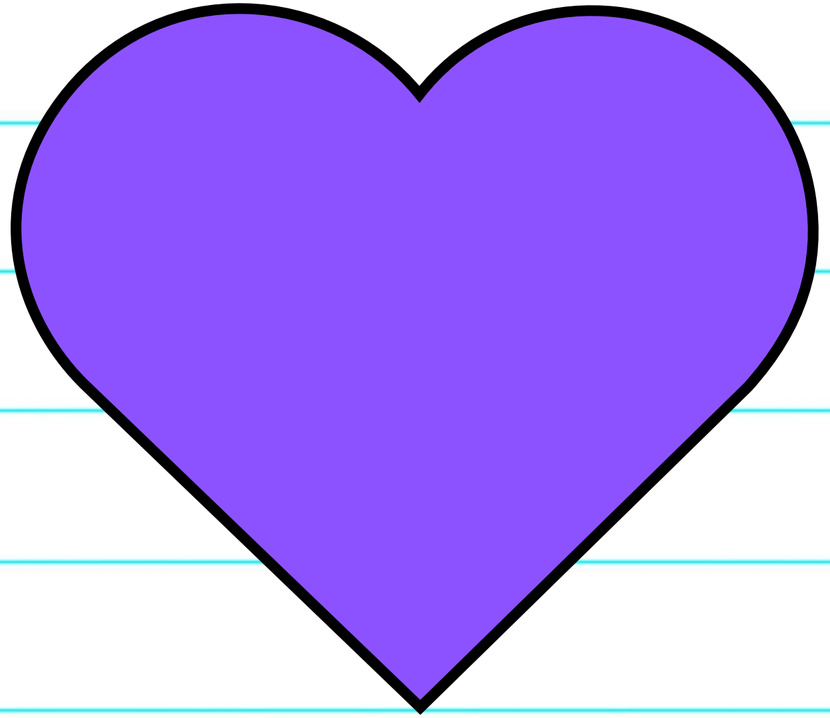
What shapes do you see?



What shapes do you see?



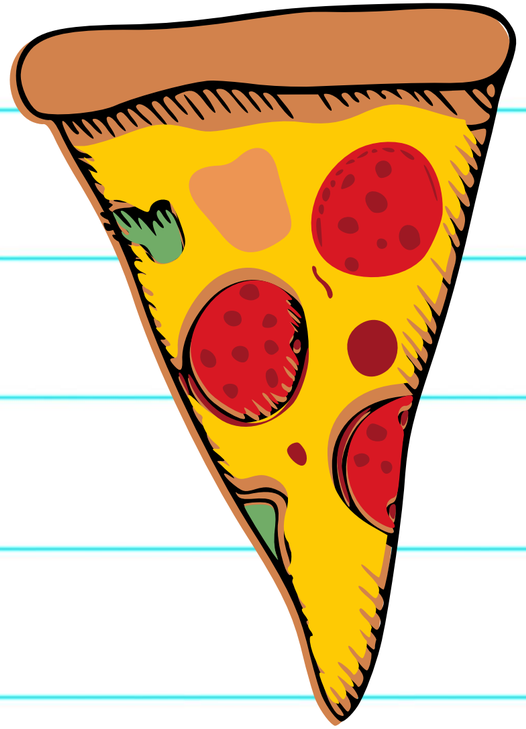
What shapes do you see?



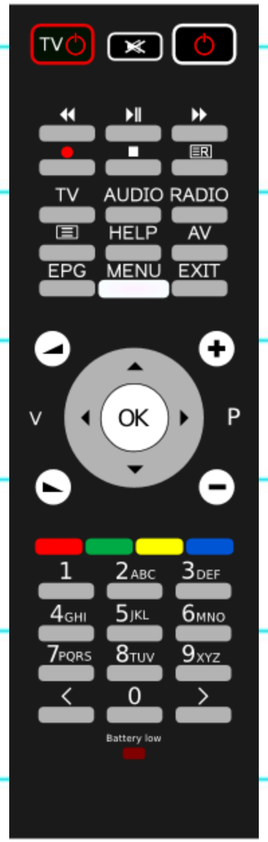
This is a heart.

What shape is the stop sign?

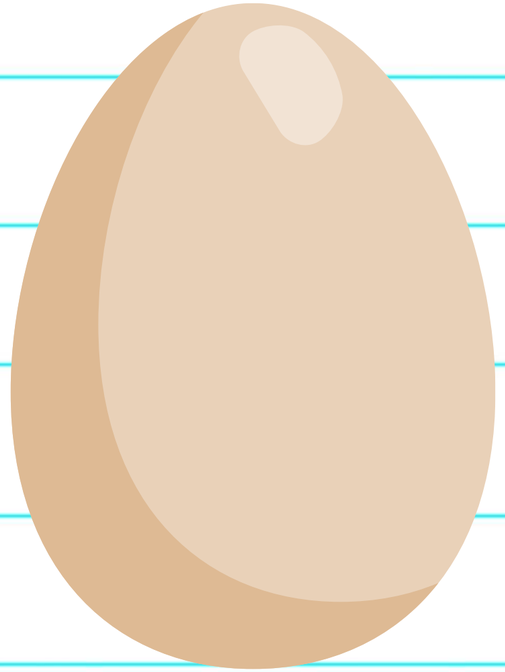
What shape is the donut?



What shape is the pizza?



What shape is the remote?



What shape is the egg?

What shape is the stamp?

LET'S DISCUSS

What was your favorite shape?